Atividade 04 - Expressões Constantes 1 - Análise Sintática

Andrei Formiga

September 21, 2026

Sumário

1. Introdução 3
2. A linguagem EC1 (Expressões Constantes 1) 4
3. Análise léxica e sintática 5
4. Análise léxica da linguagem EC1 6
   4.1. Estrutura de dados do token 6
   4.2. Espaços 6
   4.3. Exemplo de análise léxica de programa EC1 7
   4.4. Erros léxicos 7
   4.5. Comentários 8
   4.6. Uso do analisador léxico 8
5. Análise Sintática 9
   5.1. Análise sintática descendente recursiva 9
   5.2. Árvore de sintaxe abstrata 10
   5.3. Representação da árvore 11
   5.4. Análise das expressões em EC1 11
   5.5. A interface entre o analisador léxico e o sintático 12
   5.6. Um interpretador 13
   5.7. Impressão da árvore 13
6. Artefato para entrega 14
   6.1. Documentação 14

1. Introdução

Na Atividade 02 criamos o primeiro compilador da disciplina, um compilador para a linguagem de Constantes Inteiras. Em seguida, na Atividade 03 vimos como traduzir (manualmente) expressões aritméticas contendo apenas constantes para a linguagem *assembly* (x86-64 ou ARM64). Agora nosso objetivo é criar um compilador que traduzirá expressões aritméticas com operandos constantes para *assembly*.

A linguagem EC1 (Expressões Constantes 1) é um pouco mais complicada que a linguagem de Constantes Inteiras, e isso significa que o compilador para EC1 vai precisar fazer análise dos programas para poder gerar o código *assembly*.

A análise da sintaxe dos programas é normalmente feita em duas etapas: a análise léxica agrupa caracteres isolados em unidades chamadas de *tokens* que são similares às palavras da língua portuguesa. Em seguida, a análise sintática usa os *tokens* produzidos pela análise léxica para obter a estrutura sintática do programa de entrada.

Nesta atividade, o objetivo é realizar a análise léxica e a análise sintática da linguagem EC1. A geração do código *assembly* para a linguagem EC1 será foco da próxima atividade.

2. A linguagem EC1 (Expressões Constantes 1)

Um programa na linguagem EC1 é uma expressão aritmética com operandos constantes e usando as quatro operações. Todas as operações devem ser escritas entre parênteses, então não precisamos nos preocupar com precedência de operadores.

Alguns exemplos de programas na linguagem EC1 (um programa por linha):

```
333
(6 * 7)
(3 + (4 + (11 + 7)))
(33 + (912 * 11))
((427 / 7) + (11 * (231 + 5)))
```

A gramática para a linguagem EC1 é:

```
<programa> ::= <exp>
<expressao> ::= <num> | '(' <exp> <op> <exp> ')'
<op> ::= '+' | '-' | '*' | '/'
<num> ::= <digito>+
<digito> ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
```

Agora que a linguagem é mais complicada que apenas uma constante inteira, precisamos pensar em como analisar a estrutura do programa de entrada.

3. Análise léxica e sintática

O processo de analisar a estrutura de um programa de entrada para ajudar a determinar seu significado (o que o programa faz) é normalmente chamado de *análise sintática*.

A análise sintática para linguagens computacionais é tradicionalmente dividida em duas etapas:

1. a análise léxica, que consiste em agrupar os caracteres individuais em *tokens* (similares às palavras da linguagem natural) e classificar esses *tokens* em categorias sintáticas
2. a análise sintática propriamente dita, que verifica a estrutura gramatical da entrada com base na sequência de *tokens* da análise léxica

Embora seja possível fazer a análise sintática em apenas uma etapa, a divisão em análise léxica e sintática torna todo o processo muito mais fácil. Não ter a análise léxica como processo separado é como ler um texto sem separação de palavras:

> OfundadordaminhafamíliafoiumcertoDamiãoCubas,quefloresceunaprimeirametadedo-
> -séculoXVIII.Eratanoeirodeofício, naturaldoRiodeJaneiro,ondeteriamorridonapenúriae-
> naobscuridade,sesomenteexercesseatanoaria.

A base teórica para a análise léxica e análise sintática são as linguagens formais. Usamos as linguagens regulares na análise léxica, e as linguagens livres de contexto na análise sintática.

4. Análise léxica da linguagem EC1

A principal tarefa da análise léxica para a linguagem EC1 é agrupar os caracteres que formam as constantes inteiras. Um literal inteiro pode ser composto por um número arbitrário de dígitos, e a análise léxica deve reunir esses dígitos em um *token* do tipo constante.

Além das constantes, os outros tipos de *token* são relacionados à pontuação e operadores: `(`, `)`, `+`, `-`, `*` e `/`.

Ao gerar os *tokens*, a análise léxica classifica cada *token* gerado em uma das classes léxicas da linguagem. Para a linguagem EC1, as classes são:

- número
- pontuação: `(` e `)`
- operadores: `+`, `-`, `*` e `/`

A saída da análise léxica é uma sequência de *tokens*, cada um representado por alguma estrutura de dados que guarda as informações necessárias.

4.1. Estrutura de dados do token

As duas informações essenciais para guardar para cada *token* são o lexema que gerou o *token*, e a classe ou tipo do *token*. O lexema é a *string* da entrada que gerou o *token*. Por exemplo, a sequência de caracteres `1234` deve gerar um *token* do tipo número e com lexema `1234`.

Como vimos, a linguagem EC1 tem três tipos de *token*: números, pontuação e operadores. Para facilitar o uso dos *tokens* nas etapas posteriores do compilador, podemos criar um tipo separado para cada pontuação e cada operador; assim, ao invés de apenas um tipo pontuação, podemos ter um tipo para "parêntese esquerdo" e outro para "parêntese direito". Da mesma forma, podemos separar os operadores em quatro tipos. Alguns analisadores léxicos também incluem um tipo separado de *token* para sinalizar o final da entrada, muitas vezes chamado de um *token* EOF, da siga em inglês *End Of File*.

Além do lexema e do tipo do *token*, é comum guardar em cada um a posição em que ele ocorreu no arquivo de entrada. Isso é muito importante para o tratamento de erros no compilador; quando ocorre um erro no arquivo fonte, é necessário mostrar para o usuário qual o local do arquivo em que o erro foi encontrado.

A posição pode incluir apenas o deslocamento do caractere (se imaginarmos o arquivo de entrada como uma string, o deslocamento é o índice do caractere dentro dessa string), mas pode incluir também o número de linha e coluna do caractere onde ocorre o erro. Mesmo que a estrutura do *token* não guarde o número de linha e coluna, é possível obter esses números a partir do deslocamento.

4.2. Espaços

Uma função importante do analisador léxico (na maioria das linguagens) é eliminar os caracteres que contam como espaço em branco, principalmente:

1. espaço (código 32 em ASCII)
2. tabulação (código 9)

3. nova linha (código 10)
4. retorno do carro (código 13)

Existem outros caracteres considerados como espaços em branco no conjunto de caracteres ASCII e mais outros ainda no conjunto de caracteres Unicode, mas os quatro acima são os mais importantes.

Na maioria das linguagens, esses espaços entre *tokens* completos não têm nenhuma influência no significado do programa, então podem ser eliminados sem problema.

Existem linguagens como Python em que os espaços podem ter significância; no caso de Python, o nível de indentação do código determina sua estrutura. Não vamos trabalhar com linguagens que usem espaços significantes na disciplina.

4.3. Exemplo de análise léxica de programa EC1

Para o seguinte programa EC1:

```
(33 + (912 * 11))
```

A saída do analisador léxico é a seguinte sequência de *tokens*, com cada *token* seguindo o formato `<tipo, lexema, posicao>`:

```
<ParenEsq, "(", 0>
<Numero, "33", 1>
<Soma, "+", 4>
<ParenEsq, "(", 6>
<Numero, "912", 7>
<Mult, "*", 11>
<Numero, "11", 13>
<ParenDir, ")", 15>
<ParenDir, ")", 16>
```

4.4. Erros léxicos

O analisador léxico deve gerar a sequência de *tokens* correspondente ao programa de entrada. O programa de entrada do compilador é fornecido pelo usuário, e esse usuário pode cometer erros. Se o programa de entrada incluir caracteres ou sequências de caracteres que não podem ser reconhecidas como um *token* da linguagem, isso é um *erro léxico*. Se houver um ou mais erros léxicos na entrada, o analisador léxico não tem como fazer a análise léxica completa e deve reportar o erro ao usuário.

O analisador léxico pode parar o processo de análise léxica e informar o erro léxico assim que encontrar o primeiro erro léxico, ou pode tratar um erro léxico como um tipo de *token* específico e tentar continuar a análise. A segunda opção é útil pois pode reportar vários erros ao usuário de uma vez, mas pode ser um pouco mais difícil de implementar.

A posição guardada pelo analisador deve ser usada para reportar o erro, gerando mensagens como *Erro léxico na posição X*.

No caso da linguagem EC1, qualquer caractere fora do conjunto de parênteses, operadores e dígitos vai ser um erro léxico.

4.5. Comentários

Uma outra função comum do analisador léxico é remover os comentários da entrada, já que normalmente os comentários não influenciam na semântica do programa (ou seja, o que o programa faz).

Na linguagem EC1 não temos uma definição de sintaxe para comentários, mas os grupos podem adicionar suporte a comentários (e decidir a sintaxe deles) de forma opcional. Poder escrever comentários será muito útil nas linguagens mais complexas que veremos em atividades futuras.

4.6. Uso do analisador léxico

A forma de usar o analisador léxico é geralmente por uma das seguintes funções:

1. Uma função para obter o próximo *token* (por exemplo `proximo_token`) que, quando chamada, retorna o próximo *token* da entrada; quando não houver mais *tokens* na entrada, essa função sinaliza o final da entrada por algum erro, exceção, ou retornando um *token* do tipo EOF
2. Uma função para obter todos os *tokens* da entrada de uma vez em uma lista ou vetor. Neste caso não é preciso ter um *token* específico para o final da entrada, já que todos os *tokens* são colocados em uma lista.

Em compiladores usados em produção, a entrada pode ser muito grande e ter todos os *tokens* na memória ao mesmo tempo pode criar um problema de desempenho no compilador. Como normalmente a análise sintática pode prosseguir apenas olhando o próximo *token*, uma função como `proximo_token` é suficiente.

Nos compiladores que faremos nessa disciplina isso não será um problema, portanto qualquer uma das duas interfaces vai funcionar.

5. Análise Sintática

A análise sintática propriamente dita usa a sequência de *tokens* produzida pela análise léxica para determinar a estrutura do programa de entrada. Geralmente o resultado da análise sintática é uma *árvore sintática* do programa de entrada.

5.1. Análise sintática descendente recursiva

Algoritmos para fazer análise sintática são estudados desde o início da computação, o que resultou em uma grande variedade de métodos desenvolvidos para a tarefa. Alguns livros sobre compiladores dedicam centenas de páginas ao estudo da análise sintática e seus métodos.

Um dos resultados da pesquisa em análise sintática foi a criação de ferramentas chamadas *geradores de analisadores sintáticos* (*parser generators* em inglês). Esses geradores recebem uma gramática na entrada (como a gramática para a linguagem EC1 mostrada acima) e geram código (em alguma linguagem de programação) que realiza a análise sintática da entrada.

Mesmo com a existência dessas ferramentas, muitos compiladores usados hoje em dia ainda usam analisadores sintáticos feitos *à mão*, sem o uso de geradores. Existem vários fatores para isso, um dos principais sendo que é mais fácil gerar boas mensagens de erro para o programador que está usando o compilador.

Para analisadores criados diretamente, sem o uso de geradores, um dos algoritmos de análise sintática mais simples e mais utilizados é a *análise descendente recursiva*.

Na análise descendente recursiva, criamos uma função para cada não-terminal que queremos analisar. No caso da gramática para a linguagem EC1, teremos duas funções, para analisar `expressao` e `operador`. Quando um não-terminal possui mais de uma regra, deve ser possível determinar qual regra deverá ser seguida olhando apenas para o próximo *token* na entrada.

O não-terminal `expressao` tem duas regras:

```
<expressao> ::= <literal> | '(' <expressao> <operador> <expressao> ')'
```

Ao analisar uma expressão, se o próximo *token* na entrada for um literal inteiro, a primeira regra deve ser utilizada; se o próximo *token* for um parêntese abrindo, a segunda regra deve ser utilizada. Se o próximo *token* não for nem um parêntese abrindo nem um literal inteiro, o analisador encontrou um erro sintático. Portanto, a função de analisar expressões terá a seguinte forma (em pseudo-código):

```
analisaExp():
  tok = proximo_token()
  if tok.tipo == LITERAL_INTEIRO:
    # analisa constante inteira
  else if tok.tipo == ABRE_PARENTESE:
    # analisa operação binária
  else:
    # sinaliza erro sintatico
```

Entre as duas possibilidades válidas, o caso do literal inteiro é o mais simples. Como o próximo *token* é um literal, não é preciso analisar mais nada, isso já é uma expressão completa. Se a

análise sintática fosse apenas um processo de verificar se o programa possui um erro sintático ou não, bastaria retornar um valor verdadeiro nesse caso.

Mas o papel mais importante da análise sintática é determinar a estrutura do código de entrada. Nesse caso, o que a função acima deve retornar? Para capturar a estrutura do programa de entrada, nosso compilador vai retornar uma árvore sintática que será utilizada pelas etapas seguintes.

5.2. Árvore de sintaxe abstrata

Como já mencionado, a análise sintática produz na saída alguma representação intermediária do código de entrada. Uma forma comum de representação intermediária são as *árvores de sintaxe abstrata*, geralmente chamadas apenas de *árvores sintáticas*.

Uma árvore sintática é uma árvore composta por nós que descrevem as estruturas do programa de entrada; esses nós podem possuir filhos na árvore que representam sub-componentes da estrutura representada pelo nó pai.

Por exemplo, um programa na linguagem EC1 que é apenas uma constante inteira vai ser representado por uma árvore que é apenas um nó de constante inteira:

```
# codigo
42

# arvore
42
```

Se o programa for uma multiplicação, como `(7 * 6)`, a árvore seria um nó para a operação binária de multiplicação, e dois componentes que são os operandos da multiplicação:

```
# codigo
(7 * 6)

# arvore
  *
 / \
7   6
```

Vejamos a árvore para uma expressão um pouco mais complicada:

```
# codigo
(33 + (912 * 11))

# arvore
  +
 / \
33  *
   / \
 912  11
```

Note que a estrutura da árvore captura a estrutura da expressão: a expressão é uma soma tendo como operando esquerdo a constante 33, e como operando direito uma multiplicação (entre 912 e 11). Para realizar a soma, é preciso saber o valor de ambos operandos, o que

significa que a multiplicação deve ser efetuada antes da soma. Isso é o que esperamos ao ver a expressão `(33 + (912 * 11))`.

5.3. Representação da árvore

A árvore sintática pode ser representada no programa do compilador de várias formas. Em linguagens orientadas a objetos, é comum usar uma hierarquia de classes para representar a árvore.

Para a linguagem EC1, podemos definir uma classe-base `Exp` que representa expressões (e nós da árvore) em geral. Desta classe derivamos duas sub-classes: `Const` para uma constante inteira, e `OpBin` para uma operação binária. A classe `Const` precisa ter apenas um campo para o valor inteiro da constante, enquanto que `OpBin` precisa ter campos para o operador, e referências para os operandos esquerdo e direito (que são objetos da classe `Exp`). A classe `Exp` pode ser abstrata, já que nenhum objeto é criado diretamente com essa classe.

Essas classes poderiam ser definidas da seguinte forma (em pseudo-código similar a Java):

```
abstract class Exp { }

class Const extends Exp {
  int   valor;
}

class OpBin extends Exp {
  op    operador;
  Exp   esq;
  Exp   dir;
}
```

Neste código, não definimos o tipo `op` para o operador, mas esse campo apenas precisa distinguir entre as quatro operações possíveis: soma, subtração, multiplicação e divisão. Podemos usar uma enumeração ou um conjunto de constantes inteiras para isso.

5.4. Análise das expressões em EC1

Agora que sabemos como representar a árvore sintática, vamos finalizar a função de análise sintática de expressões. Se o próximo *token* for um literal inteiro, a função de análise deve criar um nó do tipo constante com o valor inteiro do literal. No código abaixo, usamos uma função chamada `inteiro` para converter da string do lexema para um valor inteiro:

```
analisaExp():
  tok = proximo_token()
  if tok.tipo == LITERAL_INTEIRO:
    return new Const(inteiro(tok.lexema))
  else if tok.tipo == ABRE_PARENTESE:
    # analisa operação binária
  else:
    # sinaliza erro sintatico
```

Para o caso do próximo *token* ser um parêntese abrindo, a análise deve seguir a produção para operações binárias:

```
<expressao> ::= '(' <expressao> <operador> <expressao> ')'
```

Ou seja, após o parêntese deve aparecer uma nova expressão (que é o operando esquerdo da operação), seguida de um operador e de uma outra expressão que é o operando direito da operação.

Após determinar que o próximo *token* é um parêntese abrindo, a análise deve reconhecer uma expressão. Como analisar uma expressão? Chamando a própria função `analisaExp` que estamos escrevendo (por isso o nome dessa técnica é análise descendente *recursiva*). Precisamos guardar o resultado dessa análise em alguma variável, depois reconhecer o operador, depois a outra expressão. Por fim, após a expressão do operando direito, é necessário que o *token* seguinte seja um parêntese fechando para concluir a expressão, caso contrário é um erro. O pseudo-código correspondente à essa descrição é:

```
analisaExp():
  tok = proximo_token()
  if tok.tipo == LITERAL_INTEIRO:
    return new Const(inteiro(tok.lexema))
  else if tok.tipo == ABRE_PARENTESE:
    esq = analisaExp()
    operador = analisaOperador()
    dir = analisaExp()
    verificaProxToken(FECHA_PARENTESE)
    return new OpBin(operador, esq, dir)
  else:
    # sinaliza erro sintatico
```

A função `verificaProxToken` verifica se o próximo *token* tem o tipo necessário (`FECHA_PARENTESE` , no caso) e sinaliza um erro caso não seja deste tipo. Se a verificação for bem sucedida, a análise continua retornando um nó que representa uma operação binária, juntando as três informações necessárias (os operandos e o operador).

A função `analisaOperador` é bem simples: verifica se o próximo *token* é um dos quatro operadores e cria um valor para representar o operador adequado; se não for um dos quatro operadores reconhecidos, é um erro sintático.

As funções `analisaExp` e `analisaOperador` fazem toda a análise sintática para a linguagem EC1.

5.5. A interface entre o analisador léxico e o sintático

Um detalhe importante nas funções de análise sintática é a interação entre o analisador sintático e o analisador léxico. Como vimos no pseudo-código acima, as funções de análise sintática acessam os *tokens* criados pela análise léxica em sequência, usando a função `proximo_token` . Essa função retorna o próximo *token* na entrada e avança a leitura da entrada para o *token* seguinte, até chegar ao fim da entrada.

Como discutido na atividade anterior, o analisador léxico pode ser organizado para retornar um *token* por vez (através da função `proximo_token`) ou pode varrer a entrada inteira e retornar uma coleção com todos os *tokens* de uma vez. Nesse segundo caso, caso a implementação não tenha uma função `proximo_token` , é fácil criar esta função a partir da coleção completa de todos os *tokens* na entrada; para isso é só lembrar qual o índice do próximo *token* na coleção; toda vez que `proximo_token` é chamada, o *token* atual da coleção é retornado, e o índice do *token* atual é incrementado.

Outra capacidade que o analisador sintático precisa é verificar se a entrada acabou. Isso pode ser feito com a função `proximo_token` retornando um *token* especial do tipo EOF (final da entrada), ou o analisador léxico pode disponibilizar uma função que retorna um valor booleano se a entrada chegou no fim ou não (`final_da_entrada`). Nesse último caso, o analisador sintático precisa verificar, antes de cada chamada a `proximo_token` , se a saída está no final ou não.

5.6. Um interpretador

A árvore sintática possui todas as informações necessárias para executar o programa EC1 (ou seja, a expressão) que foi analisado.

Executar o programa de entrada sem traduzi-lo para outra linguagem é um processo chamado de *interpretação* (ao invés de compilação). Uma forma simples de interpretador é o interpretador de varredura de árvore (*tree-walking interpreter*), que executa diretamente o programa a partir da árvore sintática.

Para um programa EC1 cuja árvore sintática é conhecida, o interpretador é um processo recursivo simples:

- Se o nó é uma constante, o valor do nó é o valor da constante
- Se o nó é uma operação binária, obtenha o valor do operando esquerdo e o valor do operando direito; o valor do nó é o valor do operando esquerdo operado com o valor do operando direito segundo o operador especificado no nó.

Se a árvore é representada com objetos de uma hierarquia de classes como sugerido acima, uma forma de implementar o interpretador é definir um método na classe base `Exp` e implementar o método nas sub-classes de maneira adequada.

5.7. Impressão da árvore

Um outro processo de varredura da árvore é a impressão, que é útil para testes. Para a linguagem EC1 podemos obter uma string similar à do programa de entrada usando um processo de impressão simples:

- Se o nó for uma constante, imprimir a constante
- Se o nó for uma operação binária, imprimir em sequência: parêntese `(` , operando esquerdo, operador direito e parêntese `)`

Outra possibilidade é imprimir a árvore em um formato mais visual, seja usando caracteres ASCII ou algum formato gráfico que possa ser transformado em imagem. Uma forma simples seria usar a ferramenta graphviz.

6. Artefato para entrega

O grupo deve entregar um programa que faz:

1. análise léxica de um programa EC1, mostrando a sequência de *tokens* correspondente à entrada
2. análise sintática de um programa EC1, mostrando a árvore sintática correspondente ao programa de entrada
3. interpreta um programa EC1, avaliando o valor da expressão e imprimindo na tela

O projeto deve incluir testes que verifiquem que os analisadores funcionem com programas corretos e programas com erros léxicos e/ou sintáticos.

Para fazer testes automatizados com o analisador léxico e sintático, é possível usar a impressão da sequência de *tokens* ou da árvore sintática como parte dos testes, por exemplo usando uma ferramenta como cram: https://bitheap.org/cram/°

6.1. Documentação

O projeto enviado também deve incluir um arquivo de documentação. A documentação deve conter, obrigatoriamente:

1. como compilar o compilador (se necessário);
2. como executar o compilador para um arquivo de entrada e gerar a sequência de *tokens*, a árvore sintática, e/ou o valor da expressão;
3. como executar os testes incluídos;
4. o grupo do projeto;
5. uma *declaração sobre o uso de LLMs* especificando como o grupo usou (ou não usou) LLMs no projeto

A documentação deve ser **sucinta**; evite enviar arquivos markdown gerados por LLMs contendo muitos detalhes desnecessários. Mandar documentação excessiva é um problema de mesma ordem de não mandar nenhuma documentação.